

```
# To have conda build upload to anaconda.org automatically, use
conda config --set anaconda_upload yes
conda upload \
    /home/codi01/anaconda3/conda-bld/linux-64/shapes-0.1-py39_0.tar.bz2
anaconda_upload is not set. Not uploading wheels: []

INFO :: The inputs making up the hashes for the build packages are as follows:
{
  "shapes-0.1-py39_0": {
    "recipe":
  }
}
```

```
=====
Resource usage summary:

Total time: 0:00:46.5
CPU usage: sys=0:00:00.2, user=0:00:00.8
Maximum memory usage observed: 31.4M
Total disk usage observed (not including envs): 244B
(base) codio@ecologyopen-metofantasy:~/workspace$ ls
shapes      to-be-updated.txt
```

```

3 meta.yaml | x
4 # name: shapes
5 # version: "0.1"
6
7 source:
8   git_rev:
9     git_url: https://github.com/codio-content/shapes-package
10
11 requirements:
12   build:
13     - python
14     - setuptools
15
16   run:
17     - python
18
19 test:
20   imports:
21     - shapes
22
23 about:
24   home: https://github.com/codio-content/shapes-package

```

[Collapse](#)

## Packages for Conda

## Conda Build

Anaconda maintains their own repository of packages. They also host packages created by their community. This brief tutorial will introduce the bare minimum to create a package ready for Conda. The code for the project is in the `shapes` package, which can be found in this [GitHub](#) repository. It should be noted that Conda expects the source code for a package to be hosted on [GitHub](#).

Start by creating a virtual environment called `build-pkg` and install the `conda-build` package as well. This package is required when building a package for Conda.

```
conda create --name build-pkg conda-build
```

Conda uses something called a "recipe" when building a package. A recipe contains three files in a dedicated folder. The files are called `build.bat` (used for Windows machines), `build.sh` (used for macOS and Linux machines), and `meta.yaml` (metadata for the package and build process). These three files are located in the directory `shapes`.

```
shapes
├── bld.bat
├── build.sh
└── meta.yaml
```

The `bld.bat` script should look exactly like this:

```
"%PYTHON%" setup.py install
if errorlevel 1 exit 1
```

While the `build.sh` script should look exactly like this:

```
$PYTHON setup.py install      # Python command to install the script.
```

These files are already created for you. You will use the same files with the same code in every Conda recipe. The `meta.yaml` file should look like the example below. This is the bare minimum of information needed to build a package. Notice the `git_url` is the URL for your package uploaded to [GitHub](#). Update the `meta.yaml` file (bottom-left) with the appropriate information.

```
package:
  name: shapes
  version: "0.1"

source:

git_rev:
git_url: https://github.com/codio-content/shapes-package

requirements:
  build:
    - python
    - setuptools

  run:
    - python

test:
  imports:
    - shapes
```

Once your recipe is complete, use the following command in the terminal (top-left). Conda is going to look in the `shapes` directory and follow the build recipe. This can take several minutes and lots of text will appear in the terminal.

```
conda-build shapes
```

After the build process is finished, you should see the following text (you may have to scroll up a bit):

```
# To have conda build upload to anaconda.org automatically, use
# conda config --set anaconda_upload yes
anaconda upload /home/codio/anaconda3/conda-bld/linux-64/shapes-0.1-py39_0.tar.bz2
```

The `anaconda upload` command will upload your package to Anaconda. This requires that you have an account for this to work. We are not going to upload our package. However, we do want to make sure the build process worked as expected. The path after `anaconda upload` is the location of the newly built package. **Important**, your location may be different than the one listed above. Be sure to copy your location and use it in the command below. Install this package with the `--use-local` flag.

```
conda install --use-local /home/codio/anaconda3/conda-bld/linux-64/shapes-0.1-py39_0
```

Use `grep` to search the list of installed packages for any that contain `"shapes"`. You should see your package installed in the virtual environment; it is ready to be used.

```
conda list | grep "shapes"
```

## Conda Forge

The Anaconda repository is divided into channels. Any packages maintained by the Anaconda company are in the official channel. If you create an account and upload your package, it will reside in your channel. `conda-forge` is a community driven channel where individuals come together to host their packages in a single channel. This is a very popular channel with high quality packages. You can read more about this channel here.

`conda-forge` has its own way of building and uploading packages. For starters, the whole process is built around [GitHub](#). You fork and clone a repository with the recipe. After filling out the recipe (including other information like documentation), you send your package as a pull request. `conda-forge` maintainers review your package, send it back for more work, or host it on their channel.

If you are interested in building Conda packages, familiarizing yourself with `conda-forge` and their unique build process is a good idea.

### Reading Question

Select all of the true statements about building packages for Conda. **Hint**, there is more than one correct answer.

- ☐ Your recipe should have three files, `bid.bat`, `build.sh`, and `meta.yaml`.
- ☐ You can upload your package to Anaconda once the build process is complete.
- ☐ The source code for the package is hosted on [GitHub](#).
- ☐ The `conda-build` command creates a wheel.

The correct answers are:

- You can upload your package to Anaconda once the build process is complete.
- Your recipe should have three files, `build.bat`, `build.sh`, and `meta.yaml`.
- The source code for the package is hosted on GitHub.

Conda does not build a wheel like Poetry and `pip`. The `conda-build` command only builds a source distribution.

Next